

JOBSHEET VI

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Inheritance

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 02 Oktober 2026

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

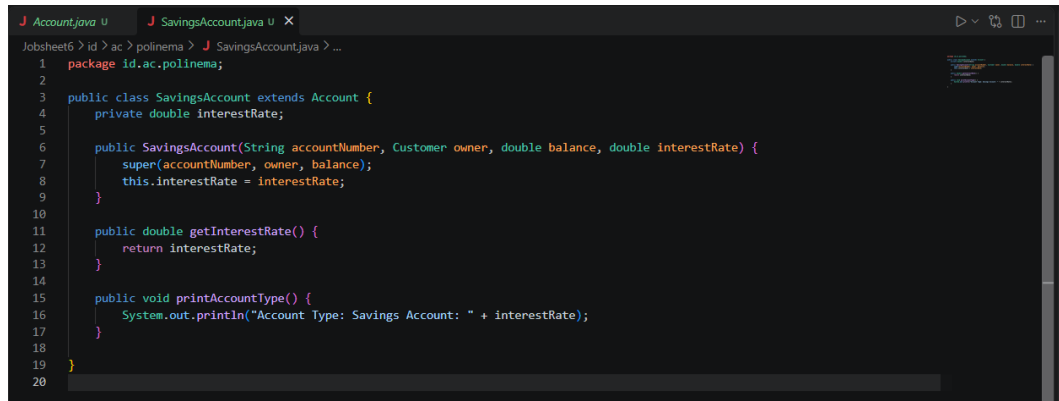
65141

TAHUN 2026-2027

C. Langkah Kerja

Langkah 1: SavingsAccount, Subclass Pertama

Code:



```
1 package id.ac.polinema;
2
3 public class SavingsAccount extends Account {
4     private double interestRate;
5
6     public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
7         super(accountNumber, owner, balance);
8         this.interestRate = interestRate;
9     }
10
11     public double getInterestRate() {
12         return interestRate;
13     }
14
15     public void printAccountType() {
16         System.out.println("Account Type: Savings Account: " + interestRate);
17     }
18 }
19
20
```

Gambar 1 SavingsAccount.java

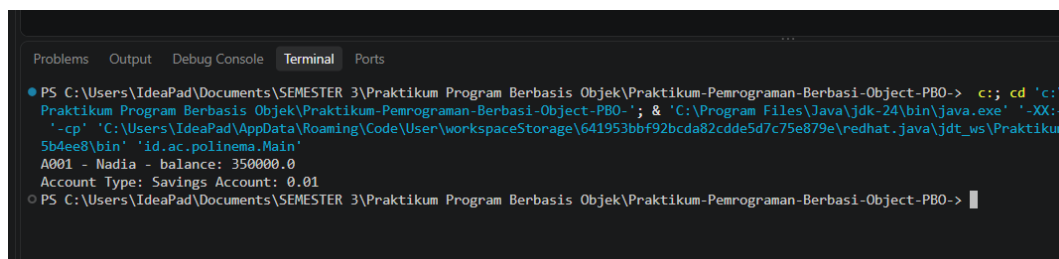
Constructor SavingsAccount memanggil super(accountNumber, owner, balance) di baris pertama untuk membangun bagian yang diwarisi dari Account, baru kemudian mengisi interestRate miliknya sendiri. Kode ini juga menambahkan getInterestRate(), method yang tidak tergambar di diagram: atribut private butuh cara dibaca dari luar kelas (kebiasaan encapsulation dari Pertemuan 3), sehingga getter seperti ini wajar ditambahkan meski tidak selalu digambar eksplisit pada setiap diagram. Perbarui Main.java untuk mengujinya:



```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
6         SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
7         acc1.withdraw(amount: 150000);
8         acc1.printInfo();
9         acc1.printAccountType();
10    }
11 }
12
```

Gambar 2 Main.java membuat SavingsAccount dan memanggil method warisan maupun method

Hasil:



```
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO> cd 'c:\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO' & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+
-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\641953bbf92bcd82cdde5d7c75e879e\redhat.java\jdt_ws\Praktikum
5b4ee8\bin' 'id.ac.polinema.Main'
A001 - Nadia - balance: 350000.0
Account Type: Savings Account: 0.01
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO>
```

Gambar 3 Hasil Langkah 1: SavingsAccount, Subclass Pertama

Penjelasan: ita membuat kelas SavingsAccount yang mewarisi (*extends*) Account. Kelas ini menambahkan atribut khusus yaitu bunga (interestRate). Konstruktor SavingsAccount memanggil super(...) untuk menginisialisasi atribut dasar (nomor rekening, nama pemilik, dan saldo) ke kelas induknya (Account), kemudian menginisialisasi interestRate. Hasil *output* menunjukkan bahwa kita bisa memanggil method withdraw() (milik Account) dan printAccountType() (milik SavingsAccount) pada objek acc1.

Langkah 2: CheckingAccount, Subclass Kedua

Code:

```
Jobsheet6 > id > ac > polinema > J CheckingAccount.java > CheckingAccount > printAccountType()
1  package id.ac.polinema;
2
3  public class CheckingAccount extends Account {
4      private double overdraftLimit;
5
6      public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
7          super(accountNumber, owner, balance);
8          this.overdraftLimit = overdraftLimit;
9      }
10
11     public double getOverdraftLimit() {
12         return overdraftLimit;
13     }
14
15     public void printAccountType() {
16         System.out.println("Account Type: Checking overdraft limit: " + overdraftLimit);
17     }
18
19 }
20
```

Gambar 4 Add CheckingAccount.java

Perbarui Main.java untuk menguji kedua jenis rekening:

```
Jobsheet6 > id > ac > polinema > J Main.java > Main > main(String[])
1  package id.ac.polinema;
2
3  public class Main {
4      public static void main(String[] args) {
5          Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
6          SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
7          acc1.withdraw(amount: 150000);
8          acc1.printInfo();
9          acc1.printAccountType();
10
11          Customer customer2 = new Customer(name: "Sari", phone: "0812-0000-0002");
12          CheckingAccount acc2 = new CheckingAccount(accountNumber: "A002", customer2, balance: 200000, overdraftLimit: 50000);
13          acc2.withdraw(amount: 230000);
14          acc2.printInfo();
15          acc2.printAccountType();
16      }
17 }
18
```

Gambar 5 Main.java menguji SavingsAccount dan CheckingAccount

Hasil:

```
PS C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\5b4ee8\bin> 'id.ac.polinema.Main'
A001 - Nadia - balance: 350000.0
Account Type: Savings Account: 0.01
A002 - Sari - balance: 200000.0
Account Type: Checking overdraft limit: 50000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
```

Gambar 6 Hasil Langkah 2: CheckingAccount, Subclass Kedua

Penjelasan: Pada Langkah 2, kita membuat kelas CheckingAccount yang juga mewarisi Account. Kelas ini memiliki atribut khusus overdraftLimit (batas penarikan melebihi saldo). Mirip dengan SavingsAccount, kita menggunakan super(...) pada konstruktornya. *Output* program menunjukkan bahwa kedua jenis rekening (SavingsAccount dan CheckingAccount) dapat berjalan dan mencetak tipe rekening serta atribut khususnya masing-masing.

Langkah 3: Bank Mini Mengelola Beberapa Jenis Rekening

Code:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
6         SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
7         acc1.withdraw(amount: 150000);
8
9         Customer customer2 = new Customer(name: "Sari", phone: "0812-0000-0002");
10        CheckingAccount acc2 = new CheckingAccount(accountNumber: "A002", customer2, balance: 200000, overdraftLimit: 50000);
11
12        Bank bank = new Bank(capacity: 10);
13        bank.addAccount(acc1);
14        bank.addAccount(acc2);
15        bank.printAllAccounts();
16    }
17 }
18
```

Gambar 7 Main.java menaruh SavingsAccount dan CheckingAccount ke dalam Bank yang sama

Hasil:

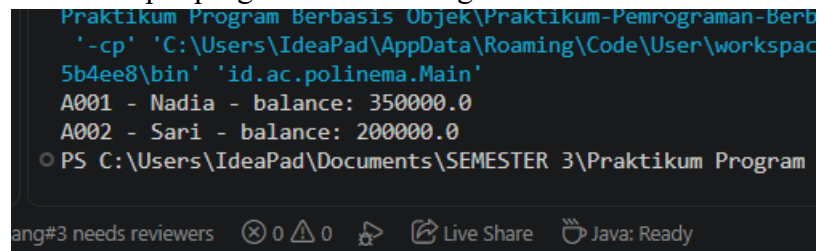
```
Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
'-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\5b4ee8\bin' 'id.ac.polinema.Main'
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
```

Gambar 8 Hasil Langkah 3: Bank Mini Mengelola Beberapa Jenis Rekening

Penjelasan: Pada Langkah 3, kita membuat objek Bank yang berfungsi untuk menyimpan berbagai jenis rekening. Karena Bank menerima parameter bertipe Account pada method addAccount(), kita bisa memasukkan SavingsAccount maupun CheckingAccount ke dalamnya. Hal ini dimungkinkan berkat konsep *upcasting*, di mana kelas anak dapat disimpan dalam referensi kelas induk. Saat kita memanggil bank.printAllAccounts(), program akan menampilkan semua info rekening yang ada di dalam *array* accounts.

D. Tugas dan Hasil Kerja

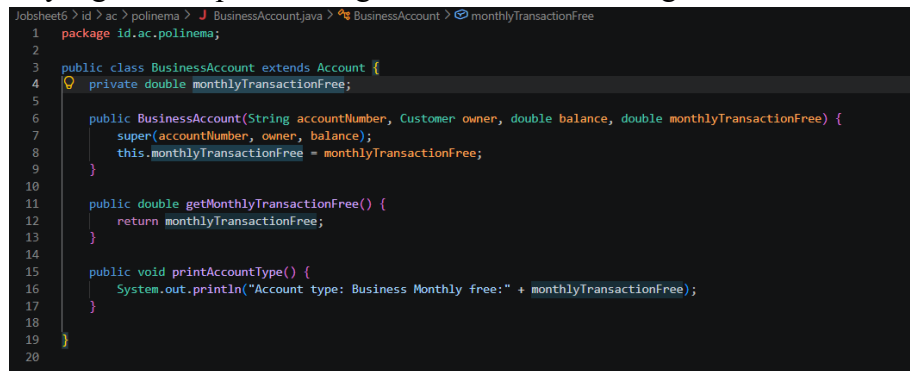
- Screenshot output program setelah Langkah 3.



```
Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berb...
'-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspac...
5b4ee8\bin' 'id.ac.polinema.Main'
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program
```

Gambar 9 Output Program Langkah 3

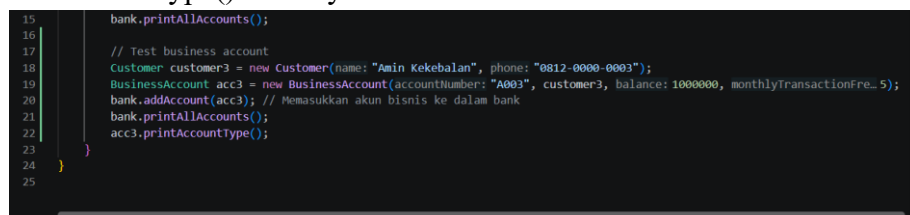
- Tugas mandiri:
1. Bank Mini menambah satu jenis rekening lagi untuk nasabah korporat. Tambahkan kelas BusinessAccount extends Account, dengan atribut monthlyTransactionFee dan method baru printAccountType(), mengikuti pola yang sama seperti SavingsAccount dan CheckingAccount:



```
Jobsheet6 > id > ac > polinema > BusinessAccount.java > BusinessAccount > monthlyTransactionFee
1 package id.ac.polinema;
2
3 public class BusinessAccount extends Account {
4     private double monthlyTransactionFee;
5
6     public BusinessAccount(String accountNumber, Customer owner, double balance, double monthlyTransactionFee) {
7         super(accountNumber, owner, balance);
8         this.monthlyTransactionFee = monthlyTransactionFee;
9     }
10
11     public double getMonthlyTransactionFee() {
12         return monthlyTransactionFee;
13     }
14
15     public void printAccountType() {
16         System.out.println("Account type: Business Monthly free:" + monthlyTransactionFee);
17     }
18
19 }
20
```

Gambar 10 BusinessAccount.java

Buktikan dengan membuat satu BusinessAccount, memasukkannya ke Bank bersama rekening lain, lalu memanggil printAllAccounts() dan printAccountType() miliknya.



```
15 bank.printAllAccounts();
16
17 // Test business account
18 Customer customer3 = new Customer(name: "Amin Kekebalan", phone: "0812-0000-0003");
19 BusinessAccount acc3 = new BusinessAccount(accountNumber: "A003", customer3, balance: 1000000, monthlyTransactionFee: 5);
20 bank.addAccount(acc3); // Memasukkan akun bisnis ke dalam bank
21 bank.printAllAccounts();
22 acc3.printAccountType();
23
24 }
25
```

Gambar 11 Main.java add to BusinessAccount and printAllAccounts() dan printAccountType()

```
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
----- Testing Business Account ---
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
A003 - Amin Kekebalan - balance: 1000000.0
Account type: Business Monthly free:5.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum P
```

Gambar 12 Hasil

Penjelasan: Menambahkan satu jenis rekening baru yaitu BusinessAccount yang mewarisi Account. Kelas ini menambahkan atribut monthlyTransactionFee (biaya transaksi bulanan). Setelah objek BusinessAccount (acc3) dibuat, kita berhasil memasukkannya ke dalam objek Bank bersama dengan jenis rekening lainnya. Saat bank.printAllAccounts() dipanggil, data acc3 ikut tercetak bersama akun-akun sebelumnya, membuktikan bahwa BusinessAccount juga diperlakukan sebagai bagian dari Account

2. Jawab secara singkat (2-3 kalimat): Bank.printAllAccounts() belum bisa menampilkan info khusus tiap jenis rekening (bunga, limit overdraft, atau biaya bulanan), padahal method printAccountType() sudah ada di setiap subclass. Mengapa demikian, dan apa yang menurutmu perlu diubah agar printAllAccounts() bisa menampilkannya secara otomatis?

Jawab: Hal ini terjadi karena *array* penyimpan pada kelas Bank menggunakan tipe data *superclass* Account, sehingga ia hanya mengenali method yang ada di Account. Agar method khusus itu bisa dipanggil otomatis dari *array*, kita perlu mendeklarasikan method printAccountType() di kelas induk Account. Nantinya, method di induk tersebut akan ditimpa (*override*) oleh masing-masing *subclass* untuk mencetak pesan spesifiknya.

3. Jawab secara singkat (2-3 kalimat): apakah SavingsAccount sebaiknya dipecah lagi menjadi subclass (misalnya jenis dengan bunga tetap dan jenis dengan bunga berjenjang)? Beri satu contoh subclass yang menurutmu masuk akal beserta atribut barunya, atau jelaskan mengapa pemecahan lebih lanjut belum diperlukan untuk Bank Mini.

Jawab:

Hasil dan Pembahasan

Program Bank Mini telah berhasil mengimplementasikan konsep *Inheritance* (pewarisan) dalam Java. Terdapat satu *superclass* yaitu Account, yang diwarisi oleh tiga *subclass*: SavingsAccount, CheckingAccount, dan BusinessAccount. Masing-masing *subclass* memanfaatkan kembali atribut dasar dari Account (nomor akun, pemilik, saldo) menggunakan kata kunci `super()`, sekaligus menambahkan atribut khasnya sendiri seperti `interestRate`, `overdraftLimit`, dan `monthlyTransactionFee`. Objek dari ketiga *subclass* ini berhasil dimasukkan dan dikelola dalam satu penampung bertipe Account pada kelas Bank.

Kesimpulan

Penggunaan pewarisan (*Inheritance*) sangat efektif untuk menghindari penulisan kode berulang (duplikasi). Atribut dan method umum cukup ditulis satu kali di kelas induk (Account), lalu diturunkan secara otomatis ke kelas-kelas anak. Kelas anak hanya perlu fokus menuliskan atribut atau perilaku khusus yang membedakannya dari kelas lain, sehingga kode menjadi lebih rapi, terstruktur, dan mudah dikembangkan di kemudian hari.

Lampiran

[Link Github Ahmad Rafid Riqkullah](https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet6/id/ac/polinema) atau <https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet6/id/ac/polinema>